# Data, hyperparameters and model performance

# Content

- Recap on machine learning
- Importance of data
- Data preparation for tabular data
  - Common issues when working with tabular data
  - The consequences of not preparing tabular data
  - The benefits of preparing tabular data
  - Data preparation steps for tabular data
    - Preparing rows/samples of data
    - Preparing columns of data
    - Normalization/Standardization
- Data preparation for images
  - Common issues when working with images
  - The consequences of not preparing image data
  - The benefits of preparing image data
  - Data preparation steps for image data
    - Resizing
    - Image to array
    - Normalization
- Overfitting
- Bias
- Data splitting
  - Stratified split
  - Cross-validation
- Data augmentation
  - Tabular data
  - Image data
    - Rotation
    - Flipping
    - Translation
    - Shearing
    - Brightness/Contrast
    - Zooming
  - Common augmentation mistakes
- Hyperparatemer search: random/grid search
- Model evaluation
- Recap for classifiers and estimators
- Introduction to tutorial data

# Recap on machine learning

Supervised vs unsupervised

- Supervised
  training samples have a known solution, e.g. which class they belong to
- Unsupervised
  training samples have no known solution and the algorithm identifies how to group samples, e.g. clustering
- Semi-supervised learning, reinforcement learning

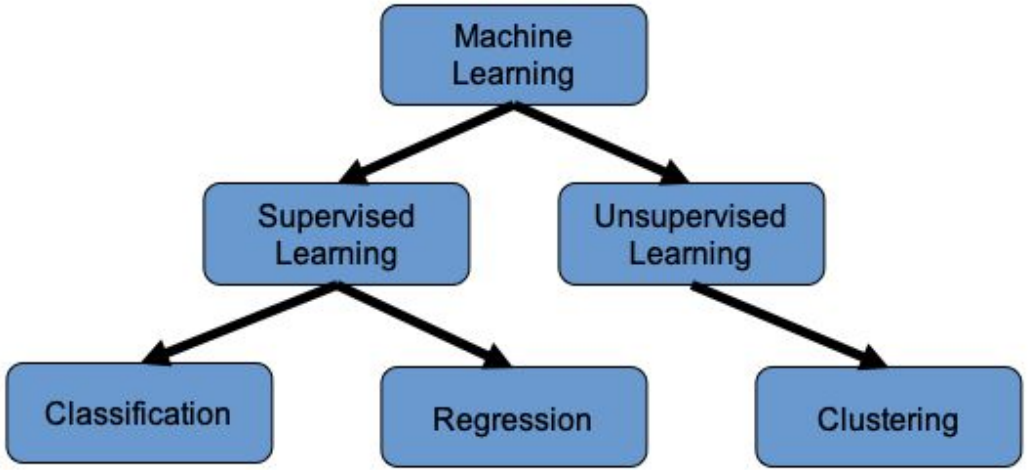
# Recap on machine learning

Classification vs regression

- Classification
  predicting/recognising two or more classes a cat, a dog, a house
- Regression
  predicting a value house price, market value at stock exchange

# Recap on machine learning

Classification vs regression
and
Supervised vs unsupervised

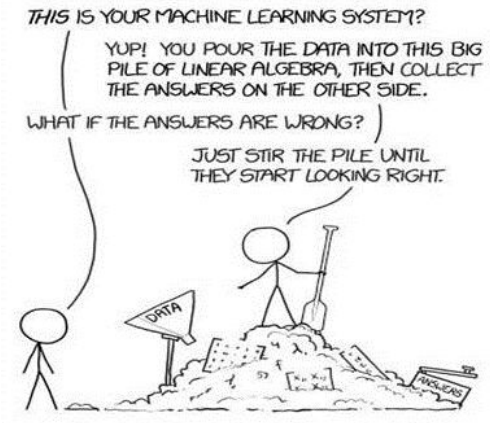


# The importance of data

Missing data
- Replace missing values by mean or 0
- Identify and apply methods for data point imputation

Standardization
- Ensure data is on same scale
- Normalize data

Exploratory data analysis
- Explore a random subset of data
- Look for gaps
- Look for bias



https://xkcd.com/1838

Data splitting
- Training and parameter search on “training set” which represents the larger part of the data overall
- Testing and assessing the trained model with “testing set” usually 20-30% of the data
- Validation set of 10-15% of the data for ultimate model validation
- Optionally a “calibration set” of 5% of the data
- Cross-validation often used with training set
- Stratification to ensure class distribution in imbalanced classes is maintained (ideally classes are equally present)
- Apply class weights during training

Your data will be biased and imbalanced. You need to consider this when interpreting results and drawing conclusions

High-quality

Diverse

For the intended application

Balanced to reduce bias

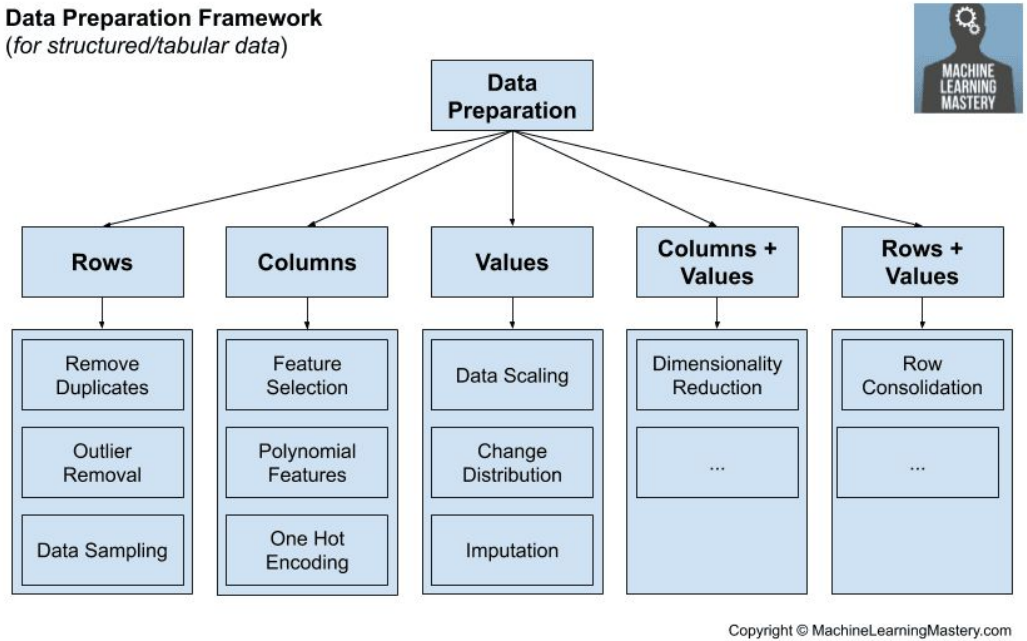
Clean/Noise-free

Benchmarking set
Publish a reusable and FAIR benchmarking set for others to use and challenge their algorithms against

# Common issues when working with tabular data

- Differ in dimensions
- Contains other data than numbers, e.g. strings/text (categorical)
- Problems with duplication
- Problems with sparsity

Unlike a human a machine/algorithm requires standardised input for best performance




https://machinelearningmastery.com/framework-for-data-preparation-for-machine-learning/

# The consequences of not preparing tabular data

- Loss of numerical stability resulting in erratic and unstable training, i.e. the loss is all over the place
- Loss doesn’t converge and training never finishes due to unscaled features cause large fluctuations in gradient steps
- Learning irrelevant, spurious features due to not encoding categorical variables (converting text categories into numerical representations) based on illogical model weights and skewed outputs
- Predictions will be essentially random
- Overfitting
- Algorithms like linear regression or neural networks cannot process raw text or missing values → runtime errors or force the algorithm to crash
- distance-based models (e.g., k-nearest neighbors or SVMs) features with larger numeric ranges (like salary in thousands) will completely overpower smaller ranges (like age) → model ignores critical variables

# The benefits of preparing tabular data

- **Prevents Misleading Weighting**
  Categorical values (e.g., colors or zip codes) must be converted into numerical formats using techniques like one-hot encoding. This prevents algorithms from assuming an ordinal relationship (e.g., that Red is "greater than" Blue)
- **Ensures Stable Gradients**
  Applying **feature scaling** (such as normalization or standardization) forces all features into a similar range (often between 0 and 1). This helps optimization algorithms like gradient descent converge much faster and prevents features with larger numerical magnitudes from dominating the model
- **Improves Generalization**
  Removing irrelevant, duplicate, or highly correlated features reduces overfitting. A well-prepared dataset allows the model to learn the underlying patterns rather than memorizing noise
- **Maintains Data Integrity**
  Handling missing values appropriately—whether through imputation (filling in gaps with means/medians) or row deletion—ensures the model receives a complete and continuous set of information during training
- **Speeds Up Training**
  Simplifying datasets, reducing dimensionality, and removing redundant columns significantly reduces the computational power and time required to train complex models

Preparing rows or samples of data / data augmentation

Data imputation

SMOTE

**Refer to data augmentation below**

Undersampling

Addressing class imbalance

Oversampling

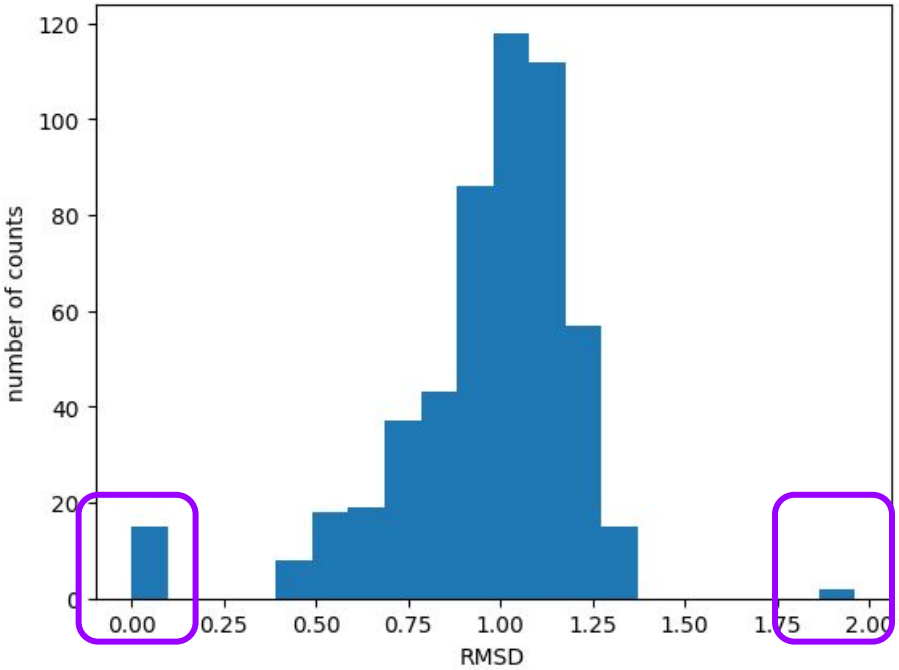
# Preparing columns of data

- Removing wrong or extreme data points
- Data plotting by feature
- One-hot-encoding of categorical data (labels)
- Recursive feature elimination
- Principal component analysis
- Identify feature correlations
- Feature importance
- Feature transformation

# Preparing columns of data

**Removing wrong or extreme data points**

Machine models perform best when data follows a Gaussian distribution



# Preparing columns of data

**Removing wrong or extreme data points**

**Standard deviation method**
mean is 50 with standard deviation 5 → data sample in range 45 and 55 → 68% of sample

- 1 Standard Deviation from the Mean: 68%
- 2 Standard Deviations from the Mean: 95%
- 3 Standard Deviations from the Mean: 99.7%

```
# identify outliers with standard deviation
from numpy.random import seed
from numpy.random import randn
from numpy import mean
from numpy import std
# seed the random number generator
seed(1)
# generate univariate observations
data = 5 * randn(10000) + 50
# calculate summary statistics
data_mean, data_std = mean(data), std(data)
# identify outliers
cut_off = data_std * 3
lower, upper = data_mean - cut_off, data_mean + cut_off
# identify outliers
outliers = [x for x in data if x < lower or x > upper]
print('Identified outliers: %d' % len(outliers))
# remove outliers
outliers_removed = [x for x in data if x >= lower and x <=
upper]
print('Non-outlier observations: %d' % len(outliers_removed))
```

# Preparing columns of data

**Removing wrong or extreme data poi**

**Inter-quartile range (IQR)**

- difference between the 75th and the 25th percentiles of the data and defines the box in a box and whisker plot
- defines the middle 50% of the data, or the body of the data

```
# identify outliers with interquartile range
from numpy.random import seed
from numpy.random import randn
from numpy import percentile
# seed the random number generator
seed(1)
# generate univariate observations
data = 5 * randn(10000) + 50
# calculate interquartile range
q25, q75 = percentile(data, 25), percentile(data, 75)
iqr = q75 - q25
print('Percentiles: 25th=%.3f, 75th=%.3f, IQR=%.3f' % (q25,
q75, iqr))
# calculate the outlier cutoff
cut_off = iqr * 1.5
lower, upper = q25 - cut_off, q75 + cut_off
# identify outliers
outliers = [x for x in data if x < lower or x > upper]
print('Identified outliers: %d' % len(outliers))
# remove outliers
outliers_removed = [x for x in data if x >= lower and x <=
upper]
print('Non-outlier observations: %d' % len(outliers_removed))
```

# Preparing columns of data

**One-hot encoding of non-numerical data (categorical)**

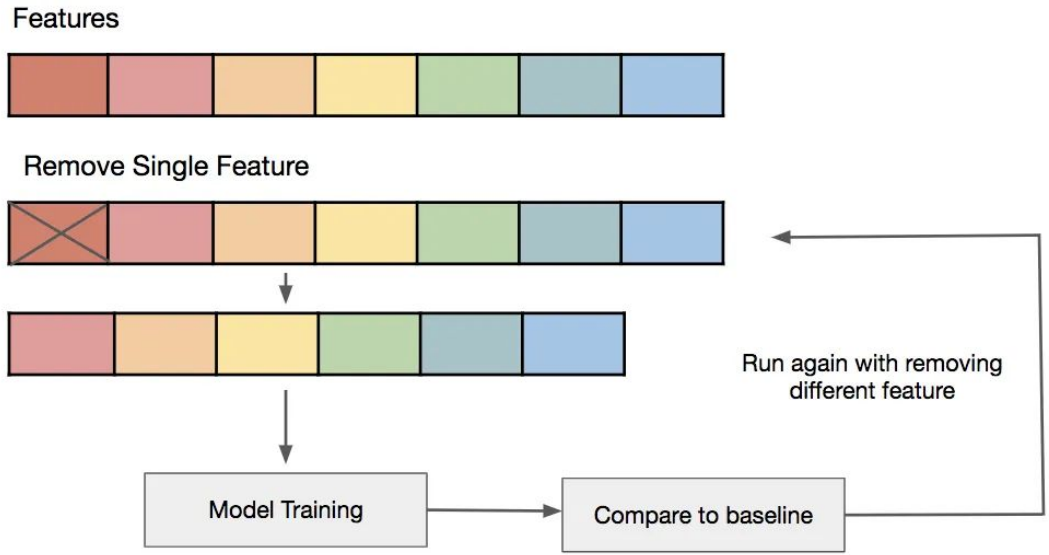
Machine models cannot “read” they can only work with numerical data

| Class label | One-hot-encoding |
|---|---|
| apple | 0 |
| pear | 1 |
| orange | 2 |
| orange | 2 |
| orange | 2 |
| apple | 0 |
| pear | 1 |
| apple | 0 |
| apple | 0 |
| pear | 1 |
| pear | 1 |
| orange | 2 |

# Preparing columns of data

**Feature selection**

- **limited amount of training samples**
- **large number of features**
- **only select the most important features for training**
- **avoid overfitting**
- **test by training individual models for these features**
- **evaluate generalization capabilities**
- **widely used feature selection methods**



# Preparing columns of data

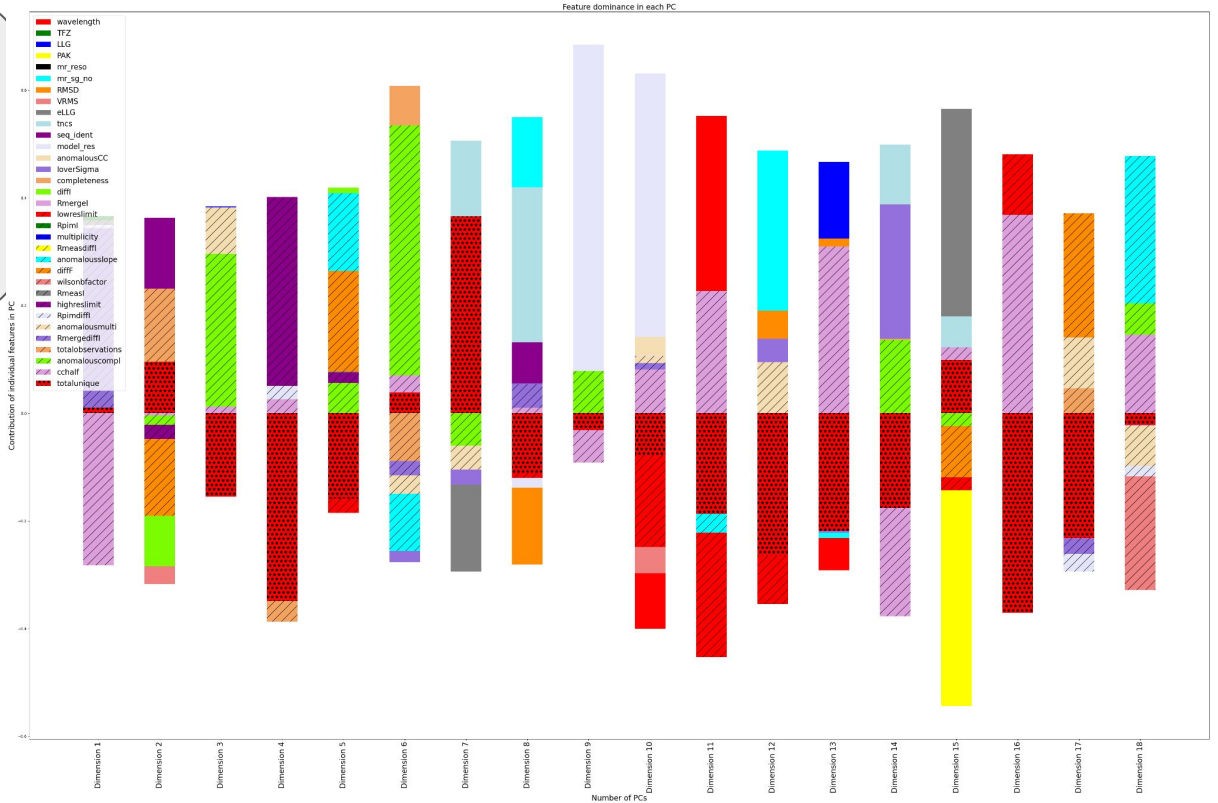
## Principal component analysis

```
#calculate the PCs to explain 95% of variance
pca = PCA(n_components=0.95)

# calculate the contributions of the different PCs
pca = PCA(n_components = max_num_pc)
 pca.fit_transform(X_train_std)
```

Max number PCs needed for 95% coverage 18

[59.87158065 49.48264985 44.83575631
38.53918975 33.68979722 31.69359513
 27.13906743 26.49755845 23.45114643
22.83209447 21.16411207 20.01416367
 19.45787382 18.09271239 17.10483684
15.72372978 15.25997847 12.90432381]



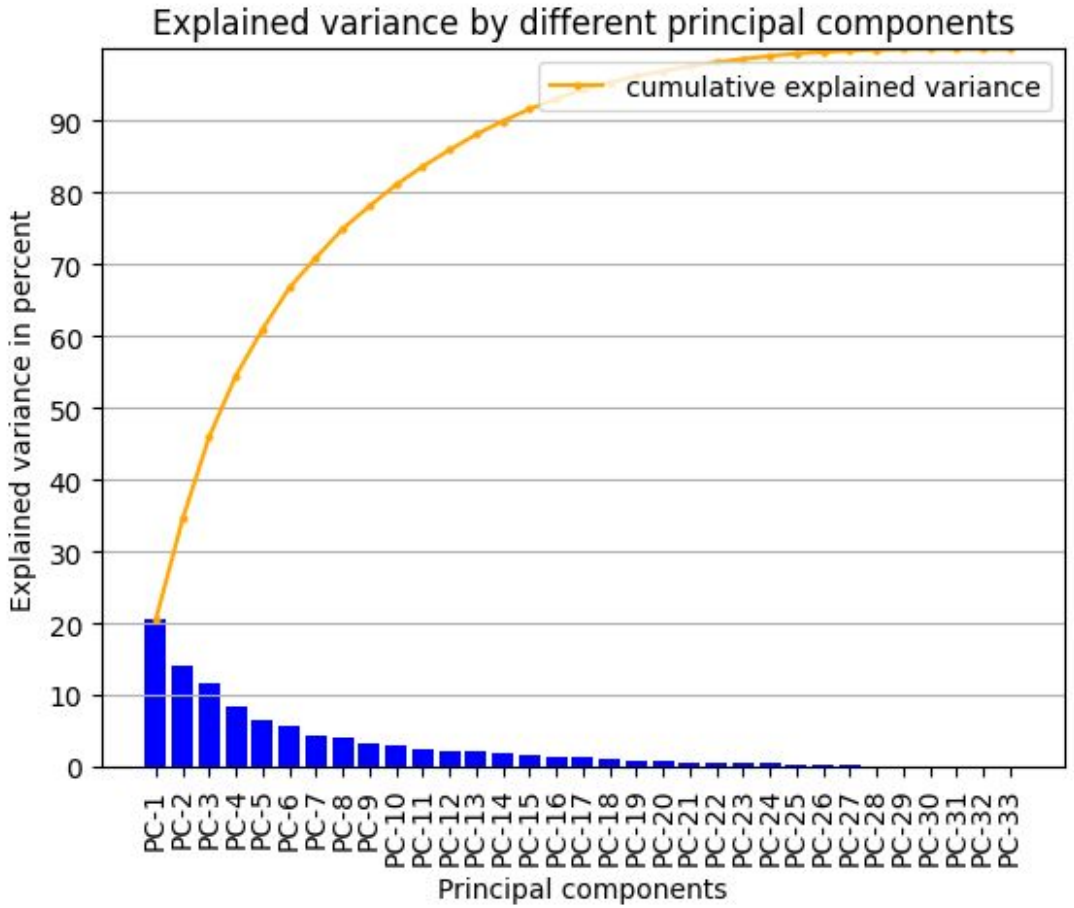
# Preparing columns of data

**Principal component analysis**

Cumsum is another way to show how many principal components explain the variance in the data

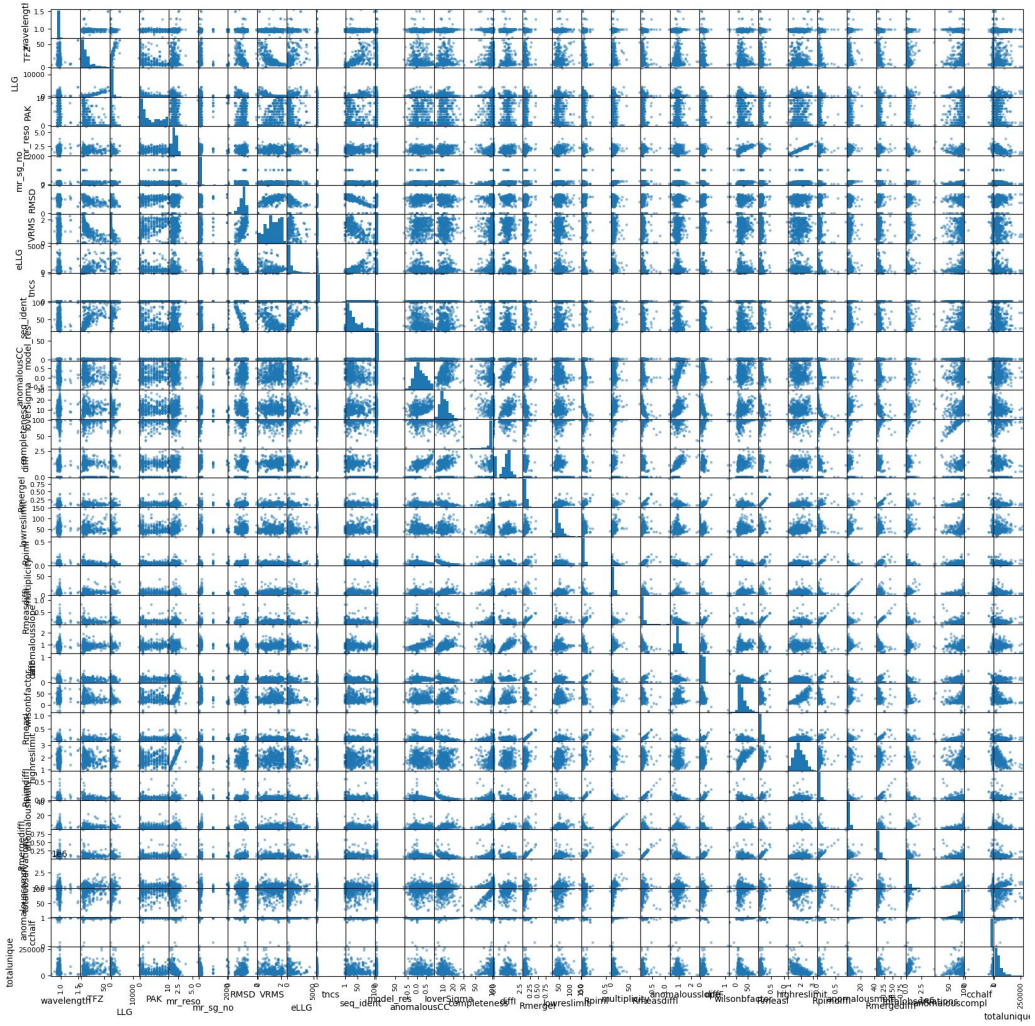
Max number PCs needed for 95% coverage 18

[59.87158065 49.48264985 44.83575631
38.53918975 33.68979722 31.69359513
27.13906743 26.49755845 23.45114643
22.83209447 21.16411207 20.01416367
19.45787382 18.09271239 17.10483684
15.72372978 15.25997847 12.90432381]



# Preparing columns of data

## Data plotting by feature



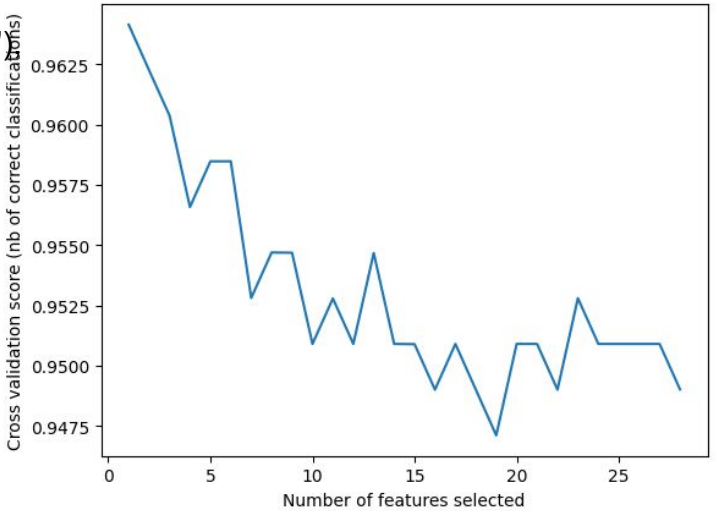
# Preparing columns of data

## Recursive feature elimination

```
svc = SVC(kernel="linear", C=0.1, class_weight='balanced')
# The "accuracy" scoring is proportional to the number of correct
# classifications
rfecv = RFECV(estimator = svc, step=1, min_features_to_select=6,
cv=StratifiedKFold(3), scoring='accuracy')
rfecv.fit(X_train_std, y_train)
```

Features sorted by their rank:

[(np.int64(1), 'LLG'), (np.int64(1), 'RMSD'), (np.int64(1), 'TFZ'), (np.int64(1), 'VRMS'), (np.int64(1), 'eLLG'), (np.int64(1), 'seq_ident'), (np.int64(2), 'PAK'), (np.int64(3), 'completeness'), (np.int64(4), 'anomalouscompl'), (np.int64(5), 'model_res'), (np.int64(6), 'tncs'), (np.int64(7), 'wilsonbfactor'), (np.int64(8), 'RpimI'), (np.int64(9), 'RmergeI'), (np.int64(10), 'cchalf'), (np.int64(11), 'anomalousCC'), (np.int64(12), 'anomalousslope'), (np.int64(13), 'RmergediffI'), (np.int64(14), 'RpimdiffI'), (np.int64(15), 'diffI'), (np.int64(16), 'lowreslimit'), (np.int64(17), 'totalobservations'), (np.int64(18), 'totalunique'), (np.int64(19), 'diffF'), (np.int64(20), 'mr_reso'), (np.int64(21), 'IoverSigma'), (np.int64(22), 'RmeasI'), (np.int64(23), 'highreslimit'), (np.int64(24), 'wavelength'), (np.int64(25), 'multiplicity'), (np.int64(26), 'anomalousmulti'), (np.int64(27), 'mr_sg_no'), (np.int64(28), 'RmeasdiffI')]
Optimal number of features : 6



# Preparing columns of data

## Feature importance

Features sorted by their rank:

[(np.int64(1), 'LLG'), (np.int64(1), 'RMSD'), (np.int64(1), 'TFZ'), (np.int64(1), 'VRMS'), (np.int64(1), 'eLLG'), (np.int64(1), 'seq_ident'), (np.int64(2), 'PAK'), (np.int64(3), 'completeness'), (np.int64(4), 'anomalouscompl'), (np.int64(5), 'model_res'), (np.int64(6), 'tncs'), (np.int64(7), 'wilsonbfactor'), (np.int64(8), 'RpimI'), (np.int64(9), 'RmergeI'), (np.int64(10), 'cchalf'), (np.int64(11), 'anomalousCC'), (np.int64(12), 'anomalousslope'), (np.int64(13), 'RmergediffI'), (np.int64(14), 'RpimdiffI'), (np.int64(15), 'diffI'), (np.int64(16), 'lowreslimit'), (np.int64(17), 'totalobservations'), (np.int64(18), 'totalunique'), (np.int64(19), 'diffF'), (np.int64(20), 'mr_reso'), (np.int64(21), 'IoverSigma'), (np.int64(22), 'RmeasI'), (np.int64(23), 'highreslimit'), (np.int64(24), 'wavelength'), (np.int64(25), 'multiplicity'), (np.int64(26), 'anomalousmulti'), (np.int64(27), 'mr_sg_no'), (np.int64(28), 'RmeasdiffI')]
Optimal number of features : 6

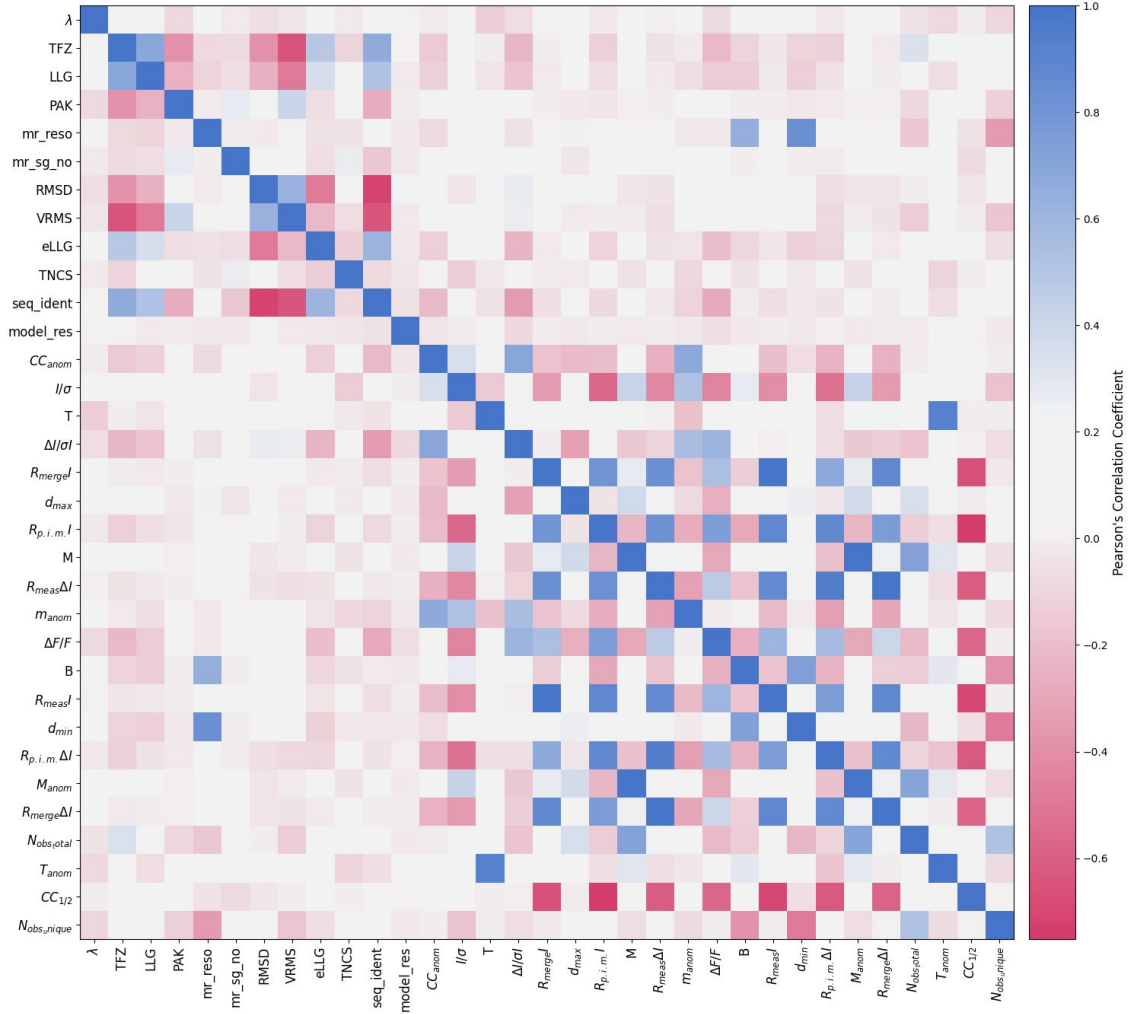
| Rank | feature |
|---|---|
| 1 | LLG, RMSD, TFZ, VRMS, eLLG, seq_ident |
| 2 | PAK |
| 3 | completeness |
| 4 | anomalouscompl |
| 5 | model_res |
| 6 | tncs |
| 7 | wilsonbfactor |
| 8 | RpimI |
| 9 | RmergeI |
| 10 | cchalf |
| 11 | anomalousCC |
| 12 | anomalousslope |
| 13 | RmergediffI |
| 14 | RpimdiffI |
| 15 | diffI |

# Preparing columns of data

## Identify feature correlations

```
corr = X_train.corr()
```

Pearson's correlation coefficient to identify correlated features



Preparing columns of data

**Feature transformation**

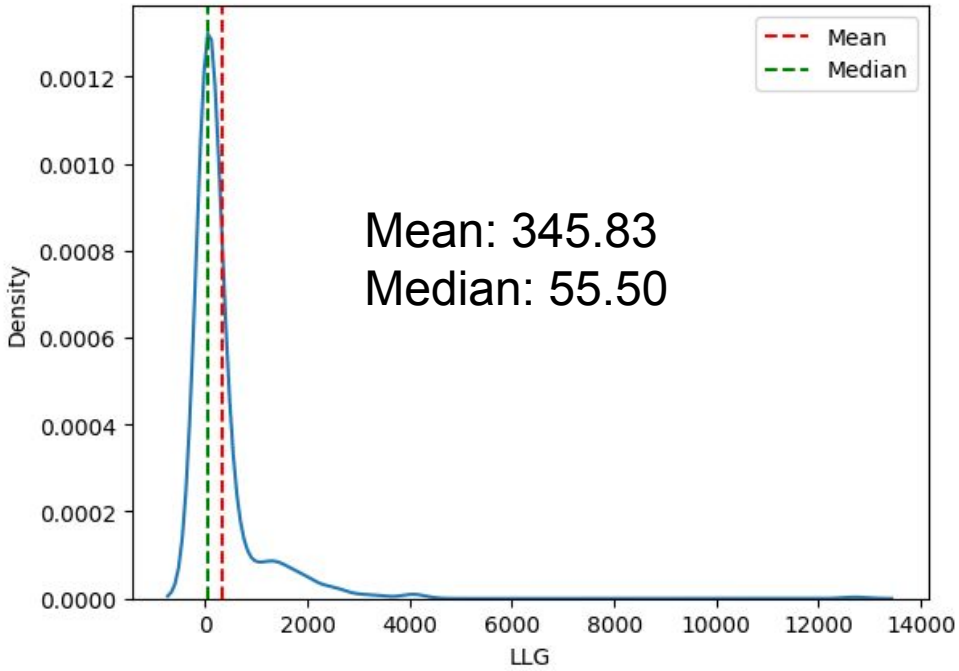
- Make your data more Gaussian-like to remove a skew
- discretization or binning change the distribution of numerical input variables
- replace missing values by imputation
- Polynomial transformation (x^1, x^2, x^3); unusual to use > 3 or 4; linear algorithms, such as linear regression and logistic regression, respond well to the use of polynomial input variables
- represent the interaction between features, e.g a new column that represents one variable multiplied by another

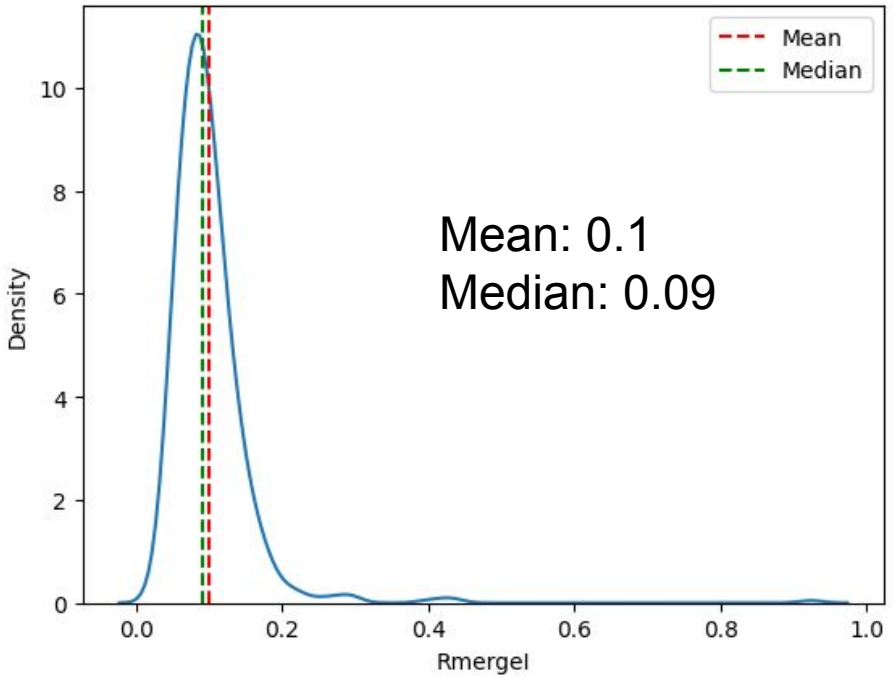
# Data scaling

**Normalizing/Standarising data**

```
X_train_std = StandardScaler().fit_transform(X_train)
```

Standardization rules are determined on the training data only



# Normalization/Standardization

```
# Initialize the StandardScaler
scaler = StandardScaler()


# Fit the scaler ONLY on the training data
scaler.fit(X_train)# Transform both training and test data
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)# Convert back to DataFrame
for easier inspection (optional)
X_train_scaled_df = pd.DataFrame(X_train_scaled,
columns=X_train.columns, index=X_train.index)
X_test_scaled_df = pd.DataFrame(X_test_scaled,
columns=X_test.columns, index=X_test.index)print("\nScaled X_train
head:\n", X_train_scaled_df.head())
print("\nScaled X_train describe:\n", X_train_scaled_df.describe())
```

https://medium.com/@morepravin1989/feature-scaling-why-and-when-to-standardize-your-machine-learning-data-721c8e411f7a

# When and when not to normalize/standardize data

- **Distance-Based Algorithms:**
  - **K-Nearest Neighbors (KNN):** Directly uses distances.
  - **K-Means Clustering:** Relies on distances for cluster assignment.
  - **Support Vector Machines (SVMs):** Especially when using RBF or polynomial kernels, which are sensitive to magnitudes.
  - **Principal Component Analysis (PCA):** PCA aims to capture maximum variance. If features are on different scales, the feature with the highest variance will dominate the principal components, even if it's less important. Standardization ensures each feature contributes proportionally.
- **Gradient Descent-Based Algorithms:**
  - **Linear Regression (with Gradient Descent)**
  - **Logistic Regression**
- **Neural Networks (Deep Learning):** The learning rates for weights would be disproportionate if features are on different scales, leading to slow or unstable convergence. Standardization helps the optimization landscape become more spherical, enabling faster convergence.
- **Regularization Techniques (L1, L2):** Regularization penalizes large coefficients. If features are not scaled, features with larger values will have naturally larger coefficients, leading to an unfair penalty. Standardization ensures all coefficients are penalized proportionally.

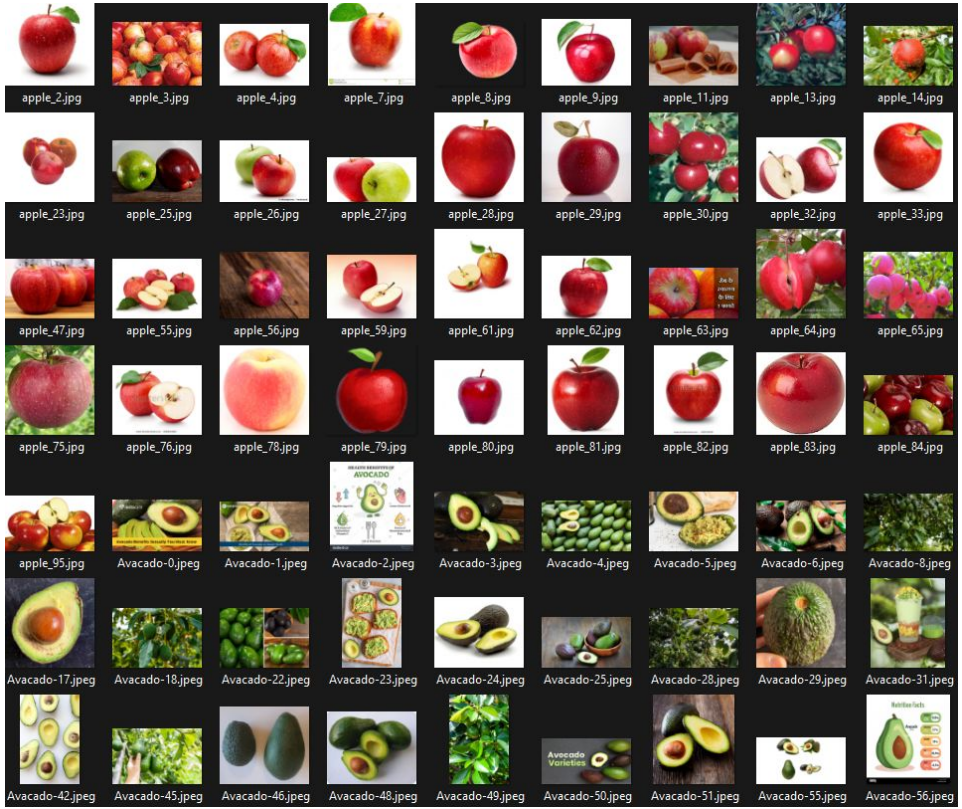
**Generally NOT Required For:**

- **Tree-Based Algorithms:**
- **Decision Trees**
- **Random Forests**
- **Gradient Boosting Machines (XGBoost, LightGBM, CatBoost)** These algorithms work by making binary splits based on threshold values (e.g., "Age > 40?"). The absolute scale doesn't affect these comparisons. While scaling won't hurt, it typically doesn't offer significant performance gains.

# Common issues when working with images

- Differ in dimensions
- Differ in resolution
- Differ in contrast
- Differ in brightness

Unlike a human a machine/algorithm requires standardised input for best performance




https://www.kaggle.com/datasets/itsmeaman03/fruit-detection-yolo

# The consequences of not preparing image data

- Most libraries will not even let you start training if images don’t have same dimensions
- Loss of numerical stability resulting in erratic and unstable training, i.e. the loss is all over the place
- Loss doesn’t converge and training never finishes
- Learning irrelevant visual biases like brightness or size
- Predictions will be essentially random
- Overfitting

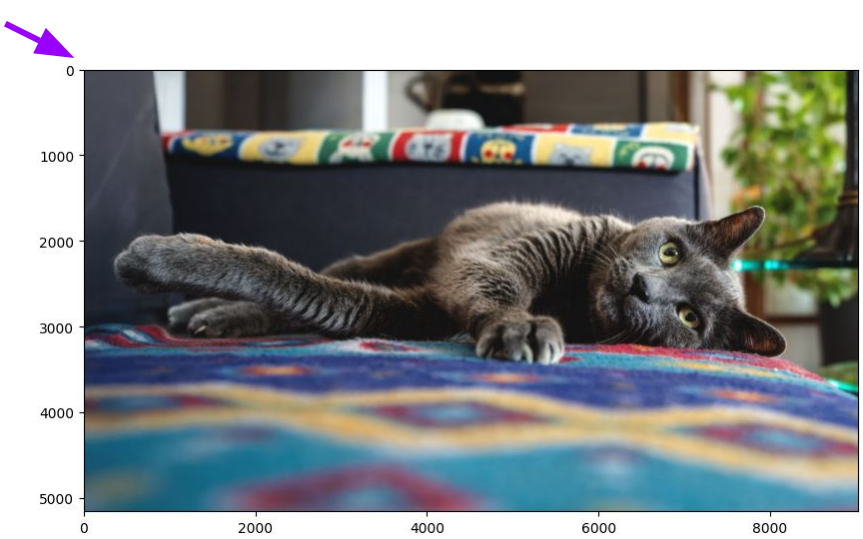
# The benefits of preparing image data

- **Faster and Stable Convergence**
  Standardizing pixels (e.g., scaling to a mean of 0 and a standard deviation of 1) ensures that the loss landscape is more spherical, allowing gradient descent to find the optimal weights directly without oscillating
- **Consistent Weight Updates**
  If images are unnormalized, pixel inputs produce huge activation values. This leads to unstable, exploding gradients that prevent the model from learning correctly with a standard learning rate
- **Preventing Mathematical Artifacts**
  Processing raw, unscaled numbers can cause floating-point precision errors in computers. Bounding values to a standard range like [0, 1] or [-1, 1] avoids this issue
- **Removing Bias Towards Ambient Properties**
  Without standardization, algorithms might unfairly lock onto image-wide characteristics (like a dark background or high contrast) rather than the actual features of the object you want to classify
- **Maintaining Spatial Uniformity**
  Resizing all images to identical dimensions allows models (which process data in rigid, uniform batches) to perform highly optimized matrix computations

# Data preparation steps for image data

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
img = load_img("pexels-rinoadamo-35224529.jpg")
plt.figure(figsize=(10,10))
plt.imshow(img)
```






https://www.pexels.com/search/cat/

# Data preparation steps for image data



- **Resizing**

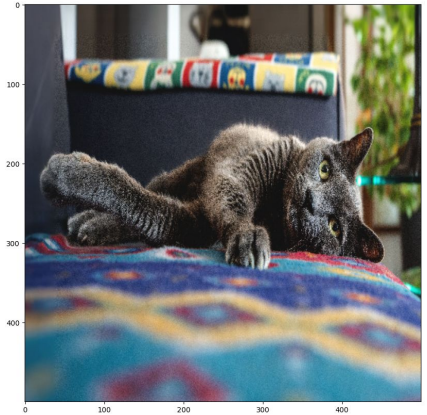
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
resized = load img("pexels-rinoadamo-35224529.jpg", target_size=(500, 500))
plt.figure(figsize=(10,10))
plt.imshow(resized)
print(resized.mode)
print(resized.size)
```

Image height

Image width

RGB

(500, 500)



# Data preparation steps for image data

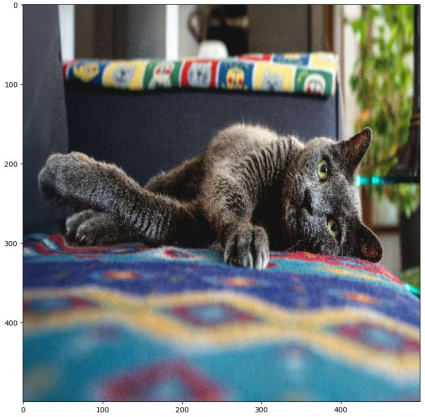


- **Image to array**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
resized_arr = img_to_array(resized)
img_resized_arr_reshaped = resized_arr.reshape((1,)+resized_arr.shape)
print(img_resized_arr_reshaped.shape)
print(img_resized_arr_reshaped[0][0])
```

Resized image to array; machine-readable format

Array to a deep-learning required format




(1, 500, 500, 3)

Batch number; here:1
→ single image

```
[[75. 77. 92.]
 [70. 73. 92.]
 [71. 74. 91.]
 ...
 [37. 49. 47.]
 [51. 72. 73.]
 [64. 84. 85.]]
```

Colour channels: 3 for RGB; 1 = greyscale

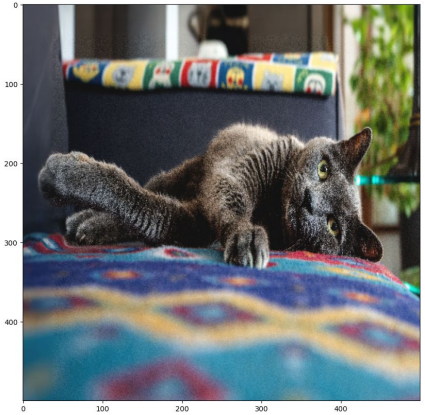
# Data preparation steps for image data



- **Normalization**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
normalize_resized_array = img_resized_arr_reshaped/255.0
print(normalize_resized_array[0][0])
```

Deep-learning ready, normalized data for training of batch size 1




Before: (1, 500, 500, 3)

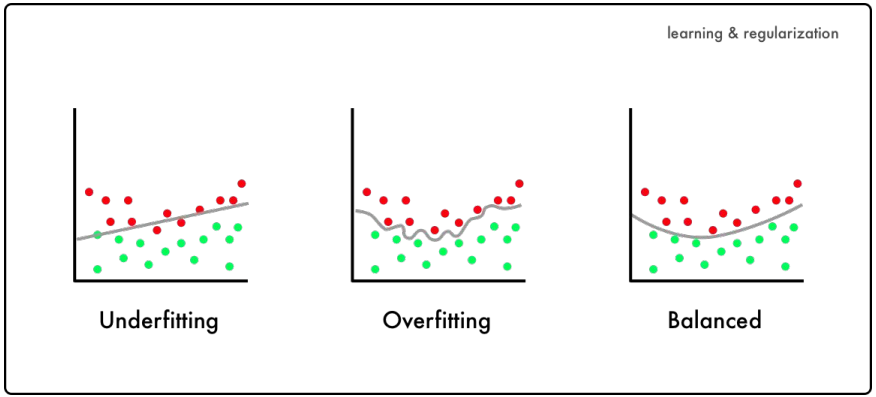
```
[[75. 77. 92.]
 [70. 73. 92.]
 [71. 74. 91.]
 ...
 [37. 49. 47.]
 [51. 72. 73.]
 [64. 84. 85.]]
```

After: (1, 500, 500, 3)

```
[[0.29411766 0.3019608  0.36078432]
 [0.27450982 0.28627452 0.36078432]
 [0.2784314  0.2901961  0.35686275]
 ...
 [0.14509805 0.19215687 0.18431373]
 [0.2        0.28235295 0.28627452]
 [0.2509804  0.32941177 0.33333334]]
```

# Overfitting

- Is when the model memorises the data
- Caused by
  - **Excessive Complexity:** The model has too much capacity (e.g., too many layers in a neural network or too deep a decision tree).
  - **Noisy Data:** The training data contains errors or irrelevant information that the model memorizes as rules.
  - **Too Little Data:** The dataset is too small to accurately represent real-world scenarios

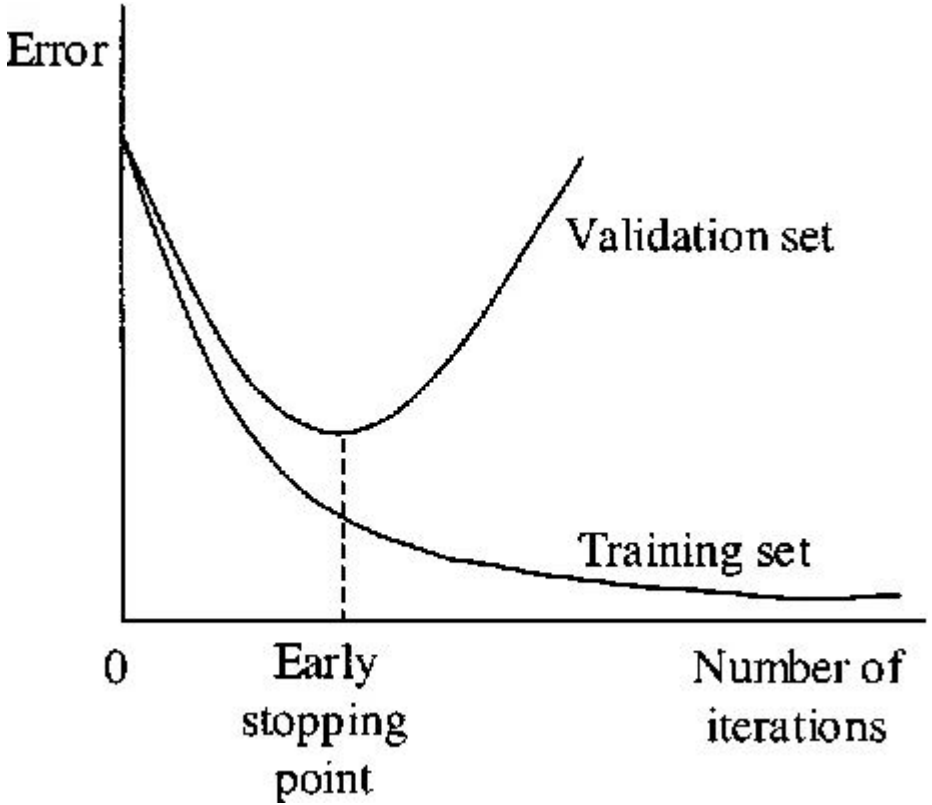


# Overfitting

Ways to address overfitting:

- **Train-Test Split & Cross-Validation**
  - Never train and test on the same data
  - Split your initial dataset into training and testing subsets to approximate for new data
  - Apply cross-validation to ensure its reliability
- **Regularization**
  - Penalize a model for being overly complex
  - L1 (Lasso) and L2 (Ridge) limit the size of model coefficients → preventing reliance on a single feature
- **Early Stopping**
  - Monitor the model's performance on a validation set as it trains
  - Stop the training as soon as the model's performance degrades regardless of training performance
- **Simplify the Model**
  - Reduce the architecture's complexity, e.g. fewer layers or hidden units in neural network or maximum depth in decision trees
- **Data Augmentation**
  - Increase data set size by generating artificial samples, e.g., rotating or flipping images, or adding slight noise)
- **Dropout**
  - in neural networks where randomly selected neurons are ignored (or "dropped") during training → prevents the network from relying too heavily on specific groups of neurons
- **Feature Selection**
  - Remove redundant, irrelevant, or noisy features from your data
  - Principal Component Analysis (PCA) to reduce the number of features while retaining the most important variances

# Overfitting



# Overfitting

**Structural Risk Minimization (SRM)**
is a framework for creating machine learning models that balance accuracy on training data with the complexity of the model itself. It prevents **overfitting**—where a model memorizes the training data but fails to generalize to new, unseen data—by penalizing overly complex functions.

- Aim is to minimize your overall "risk" (the probability of making a prediction error).
- **Hierarchy of Complexity**
  It organizes functions into nested subsets of increasing complexity (e.g., polynomials of degree 1, 2, 3, etc.)
- **Minimizing Training Error**
  It finds the hypothesis in each subset that best fits your training data
- **Balancing with a Penalty**
  It selects the optimal model by minimizing a mathematical function that combines training error and a complexity penalty → e.g. use regularisation

# Overfitting

The Mathematical Intuition
Instead of just relying on Empirical Risk Minimization (ERM), which only looks at minimizing training errors, SRM incorporates an upper bound on generalization risk based on the model's capacity. The objective formula looks like this:
Total Risk ≈ Training Error + λ × Model Complexity

- **Training Error:** How well the model performs on the data it was trained on
- **Model Complexity:** Often measured using the **VC Dimension** (Vapnik-Chervonenkis dimension), which gauges the capacity or flexibility of the model
- λ: A constant that defines how strongly to penalize complexity

SRM in Practice
You use SRM principles every time you employ **regularization**. Common real-world implementations include:

- **L1 and L2 Regularization (Ridge/Lasso)**
  These add a penalty proportional to the size of the model's weights, keeping the weights small and preventing the model from becoming too complex
- **Support Vector Machines (SVMs)**
  SVMs explicitly maximize the margin between classes while minimizing classification errors, a direct application of the SRM principle

# Bias

a systematic error that causes an algorithm to produce skewed, unfair, or discriminatory results

**What causes bias**

- **Sample Bias**

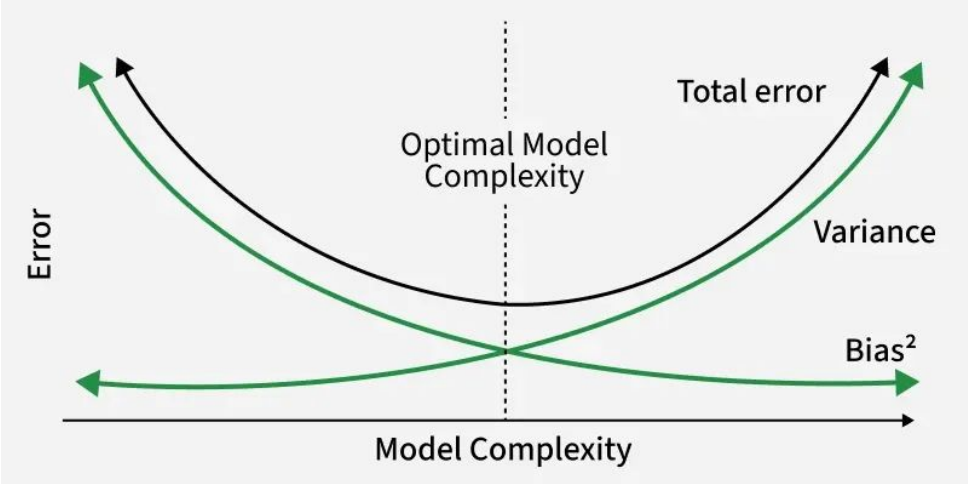
  The training data is not randomly collected, so the model only understands a specific subset of the population (e.g., training a facial recognition model mostly on lighter-skinned individuals)

- **Prejudice / Historical Bias**

  The training data contains pre-existing human stereotypes or systemic inequalities (e.g., a hiring algorithm learning to favor men because historical data shows more men were hired)

- **Proxy Bias**

  The dataset uses indirect "stand-ins" for protected characteristics (like ZIP codes acting as a proxy for race or income), causing the model to unintentionally discriminate



# Bias

a systematic error that causes an algorithm to produce skewed, unfair, or discriminatory results

**How to detect bias**

- **Exploratory Data Analysis**

  Profile your data before training to check for skewness. Look for under-representation of certain groups or missing values across demographic features

- **Subgroup Evaluation**

  Test and measure the model's performance (accuracy, false positive rates, false negative rates) individually across different demographic subgroups rather than looking at an overall average

- **Fairness Metrics & Tooling**

  Use established fairness metric toolkits to evaluate pre-training data and post-training outcomes. Tools like Microsoft Fairlearn or IBM AIF360 can automatically flag statistical disparities between privileged and unprivileged groups

# Bias

a systematic error that causes an algorithm to produce skewed, unfair, or discriminatory results

**How to address bias**

- **Curate Diverse Datasets**

  Intentionally balance your training dataset to reflect the real-world demographics of your intended user base. Over-sample underrepresented groups or collect more diverse data

- **Remove Proxy Variables**

  Scrutinize your features and remove variables that correlate too heavily with protected attributes (such as race or gender) unless absolutely critical

- **Algorithmic Adjustments**

  Apply fairness-aware machine learning during the training phase. Algorithms can be constrained to ensure "equalized odds," meaning the model yields equivalent positive/negative prediction rates across different groups

- **Continuous Post-Deployment Monitoring**

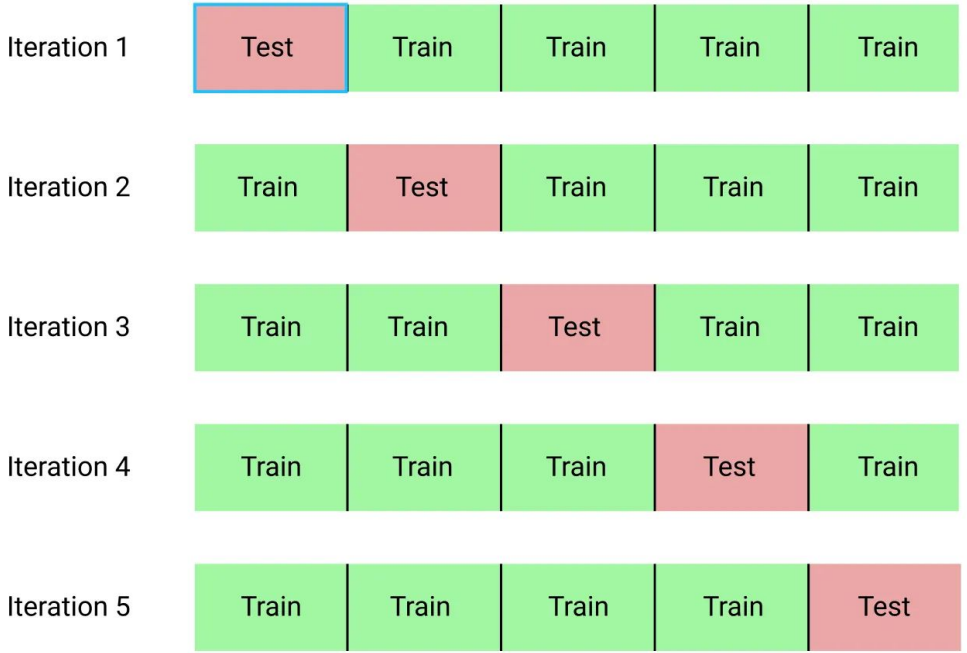
  Bias can surface after launch due to data drift. Continuously monitor model predictions in production and implement feedback loops to periodically retrain and recalibrate the system

# Data splitting

- Training and parameter search on “training set” which represents the larger part of the data overall
- Testing and assessing the trained model with “testing set” usually 20-30% of the data
- Optionally a “calibration set” of 5% of the data
- Cross-validation → often used with training set
- Stratification → to ensure class distribution in imbalanced classes is maintained (ideally classes are equally present)
- Class weights

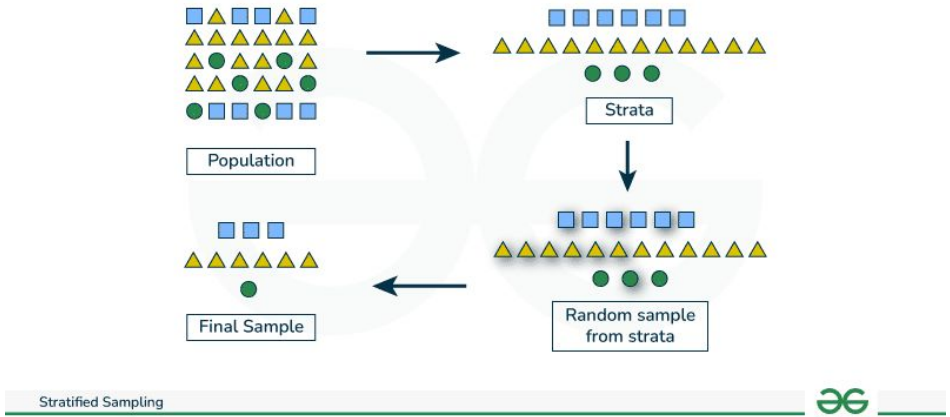
# Cross-validation

- split the dataset into *k* groups (k-fold cross-validation)
- One of the groups becomes the testing set and the others as the training set
- Repeat this process until each individual group has been used as the testing set (e.g., *k* repeats)



# Stratified data split

- a data-sampling method in machine learning that ensures the proportions of target classes or key categories in your subsets (e.g., train, validation, test) exactly match the distribution in your original dataset




```
from sklearn.model_selection import
train_test_split

X_train, X_test, y_train, y_test =
train_test_split(
    X, y,
    test_size=0.2,
    stratify=y  # <--- This ensures the target
class ratio is preserved
)
```

# Data augmentation

- a technique used in deep learning to artificially expand the size and diversity of a training dataset
- creates new, unique training examples without the need to collect or label new data

- **Reducing Overfitting**
  Exposes the model to a wider variety of representations
- **Improving Robustness**
  Helps the model recognize objects or patterns regardless of their orientation, lighting, or noise
- **Saving Costs**
  Bypasses the expensive and time-consuming process of manual data collection and annotation

# Preparing rows or samples of data / data augmentation

- **Data imputation/missing data**

**Sampling** deals with instances (rows) to correct dataset imbalances or reduce volume, whereas **imputation** deals with features (columns) to fill in missing values

- **Mean/Median/Mode**

  Replacing missing numerical values with the column's mean or median, or categorical values with the mode.

- **K-Nearest Neighbors (KNN)**

  Imputing missing features by finding similar samples in the dataset and averaging their values.

- **Multiple Imputation**

  Creating multiple plausible data versions to capture the uncertainty of missing values, often utilizing techniques like MICE.

- **Deep Learning**

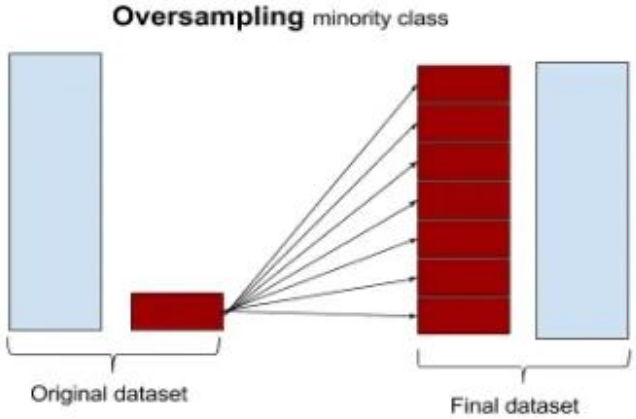
  Using autoencoders or generative models to estimate missing values in complex, high-dimensional datasets

# Preparing rows or samples of data / data augmentation

Duplicating samples from the minority class

- **Oversampling**

```
# example of random oversampling to balance
the class distribution
from collections import Counter
from sklearn.datasets import
make_classification
from imblearn.over_sampling import
RandomOverSampler
# define dataset
X, y = make_classification(n_samples=10000,
weights=[0.99], flip_y=0)
# summarize class distribution
print(Counter(y))
# define oversampling strategy
oversample =
RandomOverSampler(sampling_strategy='minority
')
# fit and apply the transform
X_over, y_over = oversample.fit_resample(X,
y)
# summarize class distribution
print(Counter(y_over))
```

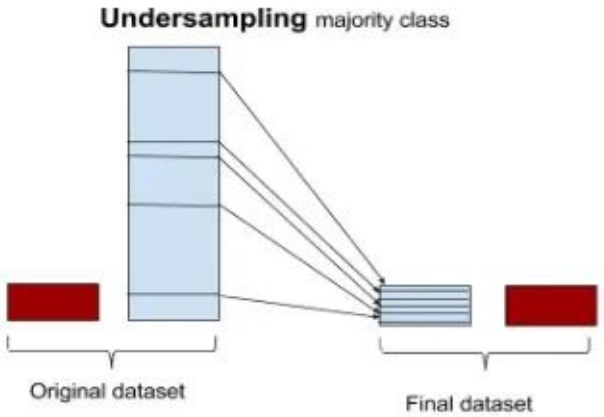


# Preparing rows or samples of data / data augmentation

Randomly removing samples from the majority class to balance a dataset

- **Undersampling/downsampling**

```
# example of random undersampling to balance
the class distribution
from collections import Counter
from sklearn.datasets import
make_classification
from imblearn.under_sampling import
RandomUnderSampler
# define dataset
X, y = make_classification(n_samples=10000,
weights=[0.99], flip_y=0)
# summarize class distribution
print(Counter(y))
# define undersample strategy
undersample =
RandomUnderSampler(sampling_strategy='majorit
y')
# fit and apply the transform
X_over, y_over = undersample.fit_resample(X,
y)
# summarize class distribution
print(Counter(y_over))
```



# Preparing rows or samples of data / data augmentation

- **Addressing class imbalance**

Applying weighting to samples depending on balance/imbalance

```
from sklearn.linear model import LogisticRegression
from sklearn.utils.class_weight import
compute class weight
import numpy as np

# Compute weights
class weights = compute class_weight('balanced',
classes=np.unique(y), y=y)
weight dict = dict(zip(np.unique(y),
class_weights))

# Use weights in the model
lr classifier =
LogisticRegression(class weight=weight_dict)
lr_classifier.fit(X_train, y_train)
```

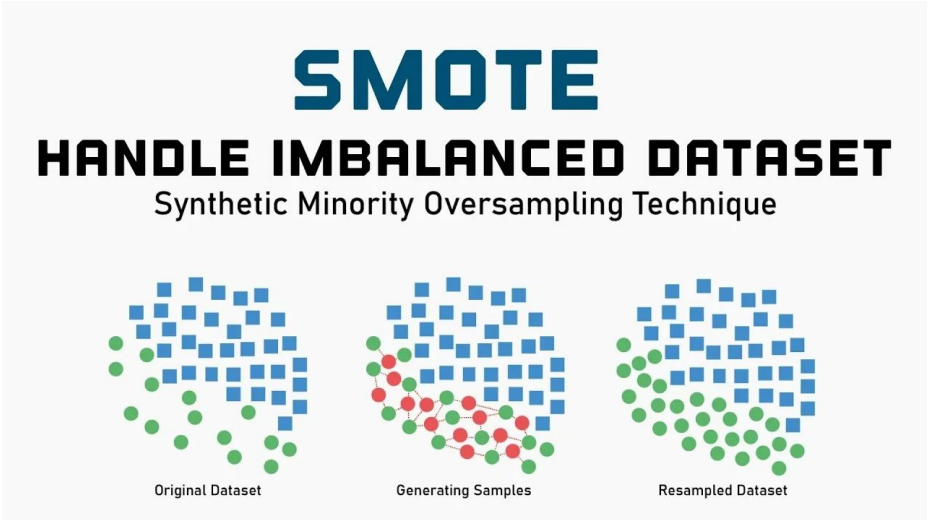
# Preparing rows or samples of data / data augmentation

Synthesizing entirely new, plausible examples of the minority class rather than just duplicating existing ones

**SMOTE (Synthetic Minority Over-sampling Technique)**

- Generates synthetic samples for the minority class using k-nearest neighbors.
- Used in imbalanced classification

```
from imblearn.over_sampling import SMOTE
from sklearn.datasets import
make_classification
# Create imbalanced dataset
X, y = make_classification(n_samples=1000,
weights=[0.9, 0.1], n_features=5)# Apply
SMOTE
smote = SMOTE(sampling_strategy='auto',
random_state=42)
X_resampled, y_resampled =
smote.fit_resample(X, y)print("Before
SMOTE:", dict(pd.Series(y).value_counts()))
print("After SMOTE:",
dict(pd.Series(y_resampled).value_counts()))
```



# Data augmentation



- **Rotation**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
datagen = ImageDataGenerator(rotation_range=40)
aug_iter = datagen.flow(normalize_resized_array, batch_size=1)
plt.figure(figsize=(10,10))
for i in range(5):
   batch=next(aug_iter)
   plt.subplot(1,5,i+1)
   plt.imshow(batch[0])
plt.axis('off')
plt.show()
```



# Data augmentation



- **Flipping - horizontal**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
datagen5 = ImageDataGenerator(horizontal_flip=True)
aug_iter5 = datagen5.flow(normalize_resized_array, batch_size=1)
plt.figure(figsize=(10,10))
for i in range(5):
 batch5=next(aug_iter5)
 plt.subplot(1,5,i+1)
 plt.imshow(batch5[0])
 plt.axis('off')
plt.show()
```



# Data augmentation



- **Flipping - vertical**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
datagen5 = ImageDataGenerator(vertical_flip=True)
aug_iter5 = datagen5.flow(normalize_resized_array, batch_size=1)
plt.figure(figsize=(10,10))
for i in range(5):
 batch5=next(aug_iter5)
 plt.subplot(1,5,i+1)
 plt.imshow(batch5[0])
 plt.axis('off')
plt.show()
```



# Data augmentation



- **Translation - width**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
datagen = ImageDataGenerator(width_shift_range=0.2)
aug_iter = datagen.flow(normalize_resized_array, batch_size=1)
plt.figure(figsize=(10,10))
for i in range(5):
   batch=next(aug_iter)
   plt.subplot(1,5,i+1)
   plt.imshow(batch[0])
   plt.axis('off')
plt.show()
```



# Data augmentation



- **Translation - height**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
datagen = ImageDataGenerator(height_shift_range=0.2)
aug_iter = datagen.flow(normalize_resized_array, batch_size=1)
plt.figure(figsize=(10,10))
for i in range(5):
   batch=next(aug_iter)
   plt.subplot(1,5,i+1)
   plt.imshow(batch[0])
   plt.axis('off')
plt.show()
```



# Data augmentation



- **Shearing**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
datagen = ImageDataGenerator(shear_range=40)
aug_iter = datagen.flow(normalize_resized_array, batch_size=1)
plt.figure(figsize=(10,10))
for i in range(5):
   batch=next(aug_iter)
   plt.subplot(1,5,i+1)
   plt.imshow(batch[0])
   plt.axis('off')
plt.show()
```



# Data augmentation



- **Brightness/Contrast**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
datagen5 = ImageDataGenerator(channel_shift_range=0.8)
aug_iter5 = datagen5.flow(normalize_resized_array, batch_size=1)
plt.figure(figsize=(10,10))
for i in range(5):
   batch5=next(aug_iter5)
   plt.subplot(1,5,i+1)
   plt.imshow(batch5[0])
   plt.axis('off')
plt.show()
```



# Data augmentation



- **Zooming**

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img, img_to_array
import matplotlib.pyplot as plt
datagen = ImageDataGenerator(zoom_range=[0.3, 1.5])
aug_iter = datagen.flow(normalize_resized_array, batch_size=1)
plt.figure(figsize=(10,10))
for i in range(5):
   batch=next(aug_iter)
   plt.subplot(1,5,i+1)
   plt.imshow(batch[0])
   plt.axis('off')
plt.show()
```





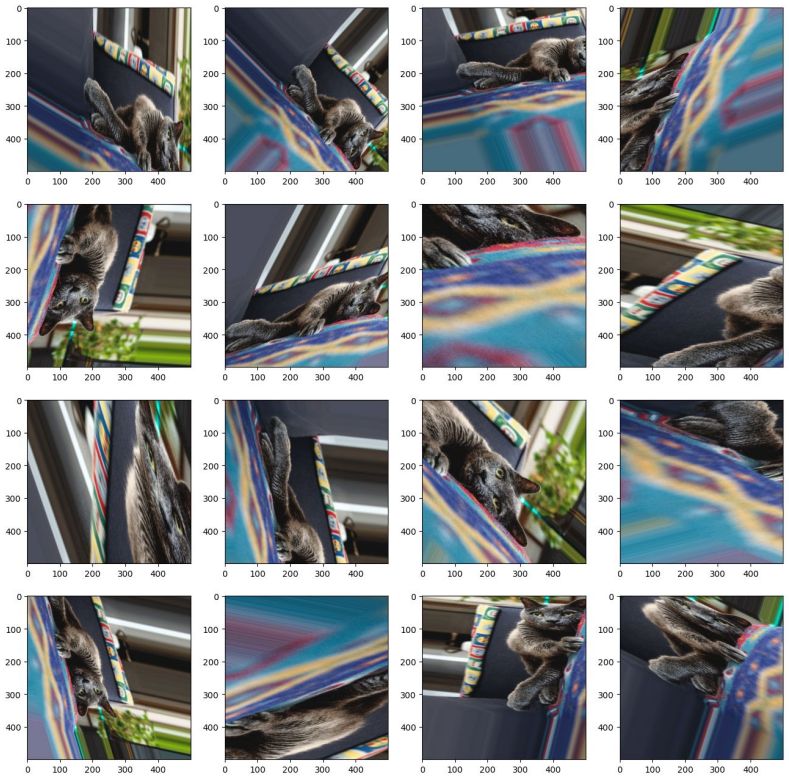


# Data augmentation

- **All augmentations combined**

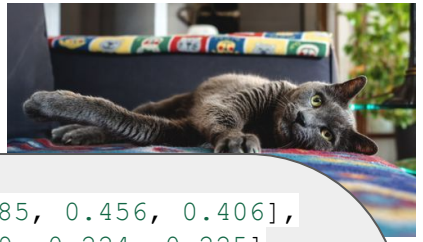
```
from tensorflow.keras.preprocessing.image
import ImageDataGenerator, load_img,
img_to_array
import matplotlib.pyplot as plt
datagen =
ImageDataGenerator(width_shift_range=[-100,
100],
                   height_shift_range=[-100,
100],
                   rotation_range=120,
                   zoom_range = [0.3, 1.5]
,shear_range=50)
#Creates our batch of one image
pic = datagen.flow(normalize_resized_array,
batch_size =1)
plt.figure(figsize=(16, 16))
#Plots our figures
for i in range(1,17):
    plt.subplot(4, 4, i)
    batch = next(pic)
    image_ = batch[0]
    plt.imshow(image_)
plt.show()
```





# Data augmentation

**tf.keras.preprocessing.image.ImageDataGenerator DEPRECATED**



- **Albuminations as alternative**

```
# Tested with albumentations>=2.0
import albumentations as A
from albumentations.pytorch import ToTensorV2
import cv2

transform = A.Compose(
    [
        A.HorizontalFlip(p=0.5),
        A.RandomBrightnessContrast(
            brightness_limit=0.7,
            contrast_limit=0.2,
            p=0.5),
        A.ShiftScaleRotate(
            shift_limit=0.3,
            scale_limit=0.05,
            rotate_limit=90,
            border_mode=cv2.BORDER_REFLECT,
            p=0.5),
        A.OneOf([
            A.GaussNoise(std_range=(0.02, 0.1), p=1.0),
            A.GaussianBlur(blur_limit=(3, 7), p=1.0),], p=0.3),
        A.CoarseDropout(
            num_holes_range=(4, 8),
            hole_height_range=(16, 32),
            hole_width_range=(16, 32),
            fill=0,
            p=0.3),
        A.Normalize(
                    mean=[0.485, 0.456, 0.406],
                    std=[0.229, 0.224, 0.225]
                ),
                ToTensorV2(),
    ],
    bbox_params=A.BboxParams(
        format="pascal_voc",
        label_fields=["class_labels"],
    ),
)

# Apply to an image with bounding boxes
image =
cv2.imread("pexels-rinoadamo-35224529.jpg")
image = cv2.cvtColor(image,
cv2.COLOR_BGR2RGB)

augmented = transform(
    image=image,
    bboxes=[[50, 60, 200, 300]],
    class_labels=["cat"],
)
augmented_image = augmented["image"]
augmented_bboxes = augmented["bboxes"]
```

# Data augmentation

- **Albuminations as alternative**



```
import numpy as np

plt.figure(figsize=(10,10))

# Convert the PyTorch tensor to a NumPy array and transpose the axes
# from (C, H, W) to (H, W, C)
image_np = augmented_image.permute(1, 2, 0).cpu().numpy()

# Define the mean and std used in the albumentations Normalize transform
mean = np.array([0.485, 0.456, 0.406])
std = np.array([0.229, 0.224, 0.225])

# Denormalize the image
image_denormalized = image_np * std + mean

# Clip the values to the valid range [0, 1] for displaying float images
image_denormalized = np.clip(image_denormalized, 0, 1)

plt.imshow(image_denormalized)
plt.axis('off')
plt.show()
```

# Common Augmentation Mistakes

**1. Augmenting validation and test sets.** Augmentation is a training-time technique only. Your validation and test sets should remain unaugmented so that metrics reflect real-world performance. The one exception is test-time augmentation (TTA), which is applied at inference with predictions averaged.

**2. Using augmentations that violate label semantics.** Flipping a "left turn" sign horizontally creates a "right turn" sign with the wrong label. Rotating digits or letters can change their meaning. Always verify that your chosen transforms preserve the ground truth.

**3. Over-augmenting.** More is not always better. Stacking too many aggressive transforms produces images so distorted that the model trains on noise. If augmented samples look unrecognizable to a human, they will confuse the model too.

**4. Ignoring annotation transforms.** Applying geometric augmentation to a detection dataset without transforming bounding boxes creates misaligned annotations. This is one of the most common bugs in custom training pipelines, and it silently degrades mAP. Use a library (Albumentations) that handles this automatically.

**5. Applying uniform augmentation to imbalanced classes.** If class A has 5,000 images and class B has 500, augmenting both equally maintains the imbalance. Apply stronger or more frequent augmentation to underrepresented classes. See this guide on solving class imbalance for deeper coverage.

https://datature.io/blog/image-augmentation-for-machine-learning-techniques-examples-code

# Hyperparameter search

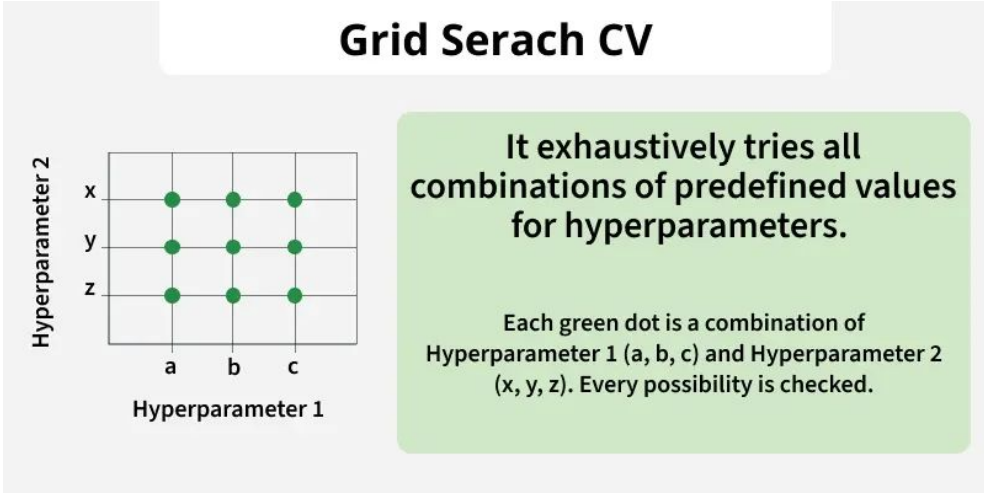
```
from sklearn.model_selection import
GridSearchCV
from sklearn.svm import SVC

# 1. Define your parameter grid
param_grid = [
  {'C': [1, 10, 100], 'kernel':
['linear']},
  {'C': [1, 10, 100], 'gamma': [0.001,
0.0001], 'kernel': ['rbf']}
]

# 2. Set up the grid search
# n_jobs=-1 utilizes all available CPU
cores to speed up training
grid_search = GridSearchCV(SVC(),
param_grid, cv=5, scoring='accuracy',
n_jobs=-1)

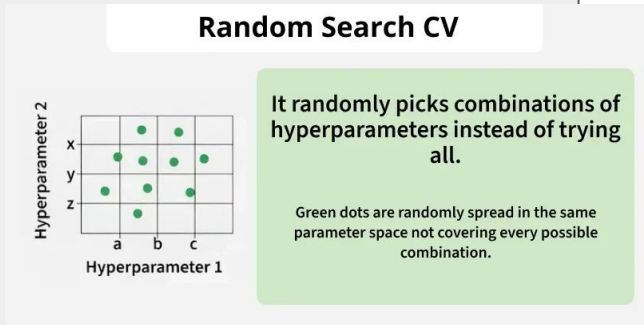
# 3. Fit the model to your data
grid_search.fit(X_train, y_train)

# 4. Access the best parameters and score
print(grid_search.best_params_)
print(grid_search.best_score_)
```



# Hyperparameter search

```
# random search logistic regression model on the sonar dataset
from scipy.stats import loguniform
from pandas import read_csv
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import RepeatedStratifiedKFold
from sklearn.model_selection import RandomizedSearchCV
# load dataset
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/sonar.csv'
dataframe = read_csv(url, header=None)
# split into input and output elements
data = dataframe.values
X, y = data[:, :-1], data[:, -1]
# define model
model = LogisticRegression()
# define evaluation
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3, random_state=1)
# define search space
space = dict()
space['solver'] = ['newton-cg', 'lbfgs', 'liblinear']
space['penalty'] = ['none', 'l1', 'l2', 'elasticnet']
space['C'] = loguniform(1e-5, 100)
# define search
search = RandomizedSearchCV(model, space, n_iter=500, scoring='accuracy', n_jobs=-1, cv=cv,
random_state=1)
# execute search
result = search.fit(X, y)
# summarize result
print('Best Score: %s' % result.best_score_)
print('Best Hyperparameters: %s' % result.best_params_)
```



# Performance indicators

Classification accuracy

$$\frac{TP + TN}{TP + TN + FP + FN}$$

Sensitivity, Recall or True Positive Rate (TPR)

$$\frac{TP}{P} = \frac{TP}{TP + FN}$$

Precision or Positive Predictive Value (PPV)

$$\frac{TP}{TP + FP}$$

F1 score

$$\frac{2}{\frac{1}{precision} + \frac{1}{sensitivity}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

False Positive Rate (FPR)

$$\frac{FP}{N} = \frac{FP}{FP + TN} = 1 - TNR$$

Specificity or True Negative Rate (TNR)

$$\frac{TN}{N} = \frac{TN}{TN + FP} = 1 - FPR$$

| | | Predicted condition | | Sources: [4][5][6][7][8][9][10][11] view · talk · edit | |
|---|---|---|---|---|---|
| | Total population = P + N | **Predicted positive** | **Predicted negative** | Informedness, bookmaker informedness (BM) = TPR + TNR − 1 | Prevalence threshold (PT) = $\frac{\sqrt{TPR \times FPR} - FPR}{TPR - FPR}$ |
| **Actual condition** | **Positive (P)** [a] | **True positive** (TP), hit[b] | **False negative** (FN), miss, underestimation | True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power = $\frac{TP}{P} = 1 - FNR$ | False negative rate (FNR), miss rate type II error [c] = $\frac{FN}{P} = 1 - TPR$ |
| | **Negative (N)** [d] | **False positive** (FP), false alarm, overestimation | **True negative** (TN), correct rejection[e] | False positive rate (FPR), probability of false alarm, fall-out type I error [f] = $\frac{FP}{N} = 1 - TNR$ | True negative rate (TNR), specificity (SPC), selectivity = $\frac{TN}{N} = 1 - FPR$ |
| | Prevalence = $\frac{P}{P + N}$ | Positive predictive value (PPV), precision = $\frac{TP}{TP + FP} = 1 - FDR$ | Negative predictive value (NPV) = $\frac{TN}{TN + FN} = 1 - FOR$ | Positive likelihood ratio (LR+) = $\frac{TPR}{FPR}$ | Negative likelihood ratio (LR−) = $\frac{FNR}{TNR}$ |
| | Accuracy (ACC) = $\frac{TP + TN}{P + N}$ | False discovery rate (FDR) = $\frac{FP}{TP + FP} = 1 - PPV$ | False omission rate (FOR) = $\frac{FN}{TN + FN} = 1 - NPV$ | Markedness (MK), deltaP (Δp) = PPV + NPV − 1 | Diagnostic odds ratio (DOR) = $\frac{LR+}{LR-}$ |
| | Balanced accuracy (BA) = $\frac{TPR + TNR}{2}$ | $F_1$ score = $\frac{2\,PPV \times TPR}{PPV + TPR} = \frac{2\,TP}{2\,TP + FP + FN}$ | Fowlkes–Mallows index (FM) = $\sqrt{PPV \times TPR}$ | *phi* or Matthews correlation coefficient (MCC) = $\sqrt{TPR \times TNR \times PPV \times NPV} - \sqrt{FNR \times FPR \times FOR \times FDR}$ | Threat score (TS), critical success index (CSI), Jaccard index = $\frac{TP}{TP + FN + FP}$ |

https://en.wikipedia.org/wiki/Precision_and_recall

# Performance indicators - working example

Classification accuracy

$$\frac{TP + TN}{TP + TN + FP + FN} \qquad \frac{41 + 59}{41 + 59 + 15 + 18} = 0.7519$$

Sensitivity, Recall or True Positive Rate (TPR)

$$\frac{TP}{P} = \frac{TP}{TP + FN} \qquad \frac{41}{41 + 18} = 0.6949$$

Precision or Positive Predictive Value (PPV)

$$\frac{TP}{TP + FP} \qquad \frac{41}{41 + 15} = 0.7321$$

F1 score

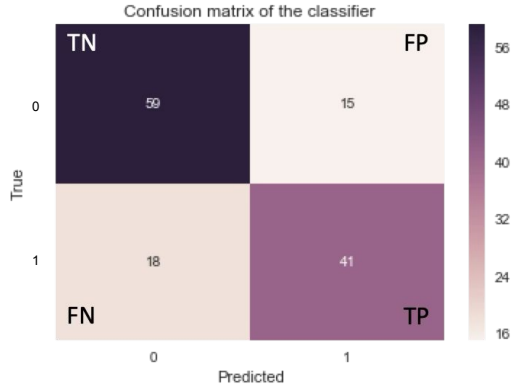
$$\frac{2}{\frac{1}{precision} + \frac{1}{sensitivity}} = \frac{TP}{TP + \frac{FN + FP}{2}} \qquad \frac{41}{41 + \frac{18 + 15}{2}} = 0.7130$$

False Positive Rate (FPR)

$$\frac{FP}{N} = \frac{FP}{FP + TN} = 1 - TNR \qquad \frac{15}{15 + 59} = 0.2027$$

Specificity or True Negative Rate (TNR)

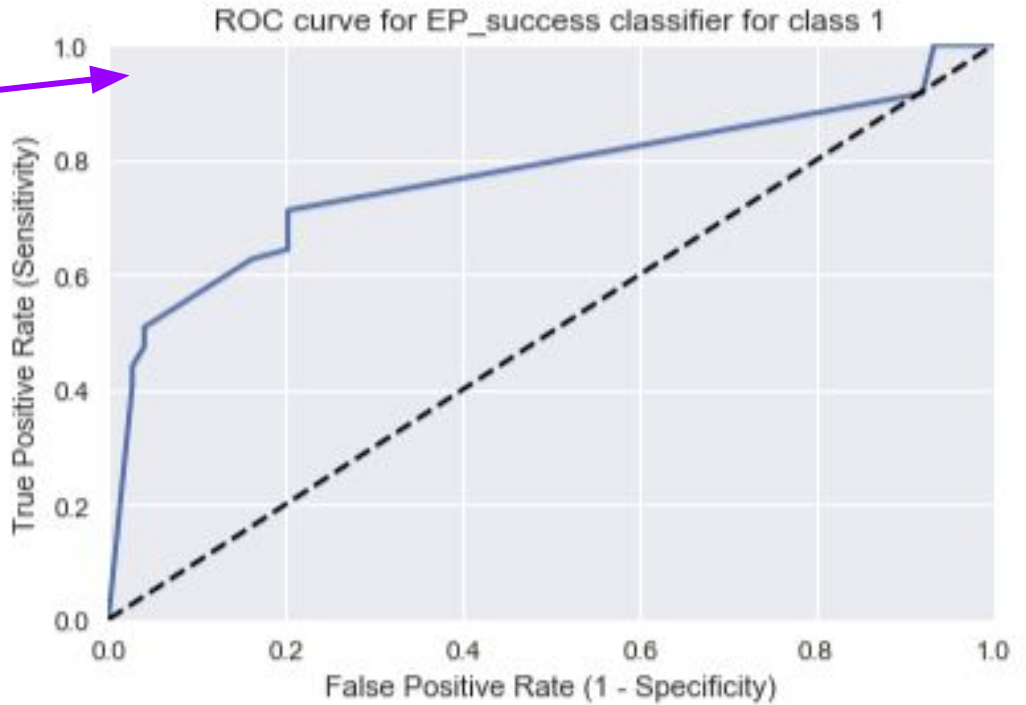
$$\frac{TN}{N} = \frac{TN}{TN + FP} = 1 - FPR \qquad \frac{59}{59 + 15} = 0.7973$$



# Performance indicators - ROC AUC

Receiver-operator curve (ROC) and area-under-the-curve (AUC)

Curve should be here




AUC for test set class 1:
0.769697663765

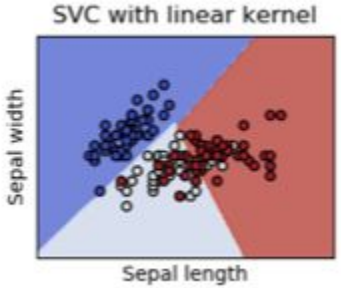
# Classifiers and estimators

- **Support vector machine with linear kernel**
- **Support vector machine with radial-base function kernel**
- **Decision tree**
- **Decision tree with Bagging**
- **Decision tree with AdaBoost**
- **Random forest**
- **Extreme random forest**
- **K nearest neighbor**

# Classifiers and estimators

**Support vector machine with linear kernel**

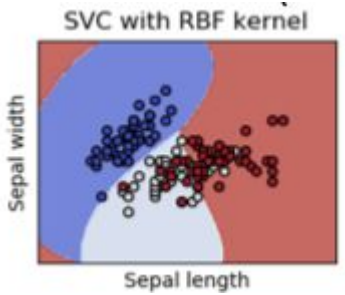
- **Separate samples by linear decision boundary**
- **Samples closest to the boundaries define the boundaries → memory efficient**
- **Maximum separation between classes/groups of samples has to be achieved**
- **Works well with high-dimensional data, i.e. large number of features compared to number of samples**
- **Very large number of features à prone to overfitting**
- **Getting probability estimates is computationally expensive**
- **Data needs to be scaled/standardized (same scaling for training and test data)**



# Classifiers and estimators

**Support vector machine with radial-base function kernel**

- **Separate samples by non-linear decision boundary**
- **Samples closest to the boundaries define the boundaries → memory efficient**
- **Maximum separation between classes/groups of samples has to be achieved**
- **Works well with high-dimensional data, i.e. large number of features compared to number of samples**
- **Very large number of features à prone to overfitting**
- **Getting probability estimates is computationally expensive**
- **Data needs to be scaled/standardized (same scaling for training and test data)**



# Classifiers and estimators



**Decision tree**

- **Simple to understand and to interpret; human readable**
- **Requires little data preparation; no support for missing values**
- **Able to handle both numerical and categorical data**
- **Able to handle multi-output problems**
- **White box model. If a given situation is observable in a model, the explanation for the condition is easily explained by boolean logic; black box model (e.g. in an artificial neural network), results may be more difficult to interpret.**
- **Possible to validate a model using statistical tests → reliability of the model**
- **Overfitting by over-complex trees that do not generalise the data well**
- **Decision trees instability → ensemble**
- **Decision tree learners create biased trees for dominant classes**

# Classifiers and estimators

**Decision tree with Bagging**

- **Contains several instances of a black-box estimator on random subsets of the original training set**
- **Aggregation of their individual predictions to form a final prediction**
- **Reduce the variance of a base estimator (e.g. a decision tree), by introducing reduce overfitting**
- **work best with strong and complex models (e.g. fully developed decision trees), in contrast with boosting methods which usually work best with weak models (e.g. shallow decision trees)**
- **Here: we use replacement**

# Classifiers and estimators

**Decision tree with AdaBoost**

- **sequence of weak learners (i.e. models that are only slightly better than random guessing, such as small decision trees)**
- **Used until all samples**
- **applying weights after each round to incorrectly predicted samples**
- **Continue until all samples classified**
- **Each subsequent weak learner is thereby forced to concentrate on the examples that are missed by the previous ones in the sequence**

# Classifiers and estimators

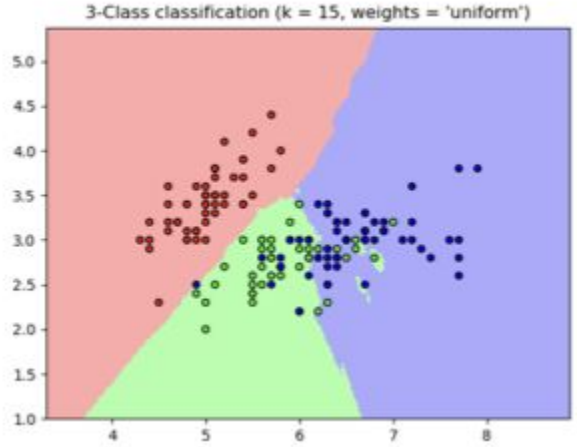
**Random forest**

- **each tree in the ensemble is built from a sample drawn with replacement (i.e. a bootstrap sample) from the training set**
- **splitting each node either from all input features or a random subset of size max_features**
- **Two sources of randomness à decrease the variance of the forest estimator**
- **taking an average of those predictions à reduced variance, sometimes at the cost of a slight increase in bias**
- **averaging their probabilistic prediction, instead of letting each classifier vote for a single class**

# Classifiers and estimators

**K nearest neighbor**

- **Neighbors-based classification**
- **instance-based learning or non-generalizing learning à does not learn a general rule to identify the classes**
- **Classification is computed using simple majority à a query point is assigned the data class which has the most representatives within the nearest neighbors of the point**



# Classifiers and estimators

**Extreme random forest**

- **More randomness in the way splits are computed**
- **Random subset of candidate features is used**
- **Discriminative thresholds are drawn at random for each candidate**
- **reduce the variance of the model a bit more, at the expense of a slightly greater increase in bias**

# Tutorial data

JCSG → 507 structures
SGC → 303 structures

Phasing method:
S/MAD → 446
Native → 364

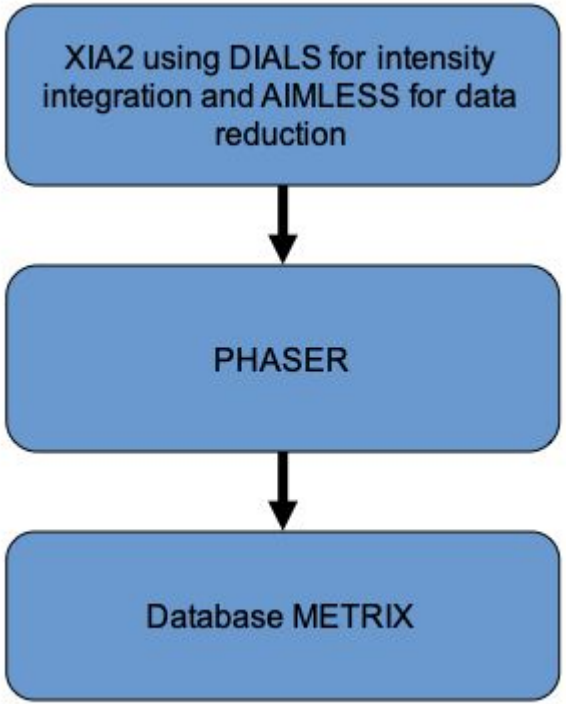
Training data:
Samples 664

Resolution range:
1.05 – 3.8Å

Detector type:
CCD, PAD

X-Ray source:
Synchrotron, in-house

Protein:
6 – 100kDa



# Further reading and resources

https://machinelearningmastery.com/feature-selection-with-real-and-categorical-data/
https://machinelearningmastery.com/how-to-use-statistics-to-identify-outliers-in-data/
https://machinelearningmastery.com/framework-for-data-preparation-for-machine-learning/
https://machinelearningmastery.com/feature-extraction-on-tabular-data/
https://medium.com/@kalanarandil/image-preprocessing-for-machine-learning-a-deep-dive-603f52c56ed1
https://github.com/IBM/image-preprocessing-for-deep-learning-models/blob/master/README.md
https://www.tensorflow.org/tutorials/load_data/images
https://machinelearningmastery.com/best-practices-for-preparing-and-augmenting-image-data-for-convolutional-neural-networks/
https://towardsdatascience.com/image-data-augmentation-for-deep-learning-77a87fabd2bf/
https://www.tensorflow.org/tutorials/images/data_augmentation
https://www.kaggle.com/datasets/itsmeaman03/fruit-detection-yolo
https://www.geeksforgeeks.org/computer-vision/what-is-data-augmentation-how-does-data-augmentation-work-for-images/
https://medium.com/@gmshakil786/data-augmentation-in-deep-learning-the-complete-beginners-guide-76220cba3d7d
https://www.ibm.com/think/topics/data-augmentation
https://www.geeksforgeeks.org/machine-learning/python-data-augmentation/
https://www.geeksforgeeks.org/electronics-engineering/geometric-transformation-in-image-processing-1/
https://datature.io/blog/image-augmentation-for-machine-learning-techniques-examples-code

# Further reading and resources

https://medium.com/@tubelwj/five-methods-for-data-splitting-in-machine-learning-27baa50908ed
https://www.geeksforgeeks.org/machine-learning/splitting-data-for-machine-learning-models/
https://www.kaggle.com/discussions/general/448072
https://medium.com/@morepravin1989/feature-scaling-why-and-when-to-standardize-your-machine-learning-data-721c8e411f7a
https://medium.com/data-science/8-simple-techniques-to-prevent-overfitting-4d443da2ef7d
https://www.geeksforgeeks.org/machine-learning/data-imputation-techniques-in-ml/
https://www.geeksforgeeks.org/machine-learning/bias-vs-variance-in-machine-learning/
https://scikit-learn.org/stable/modules/impute.html
https://python.plainenglish.io/balancing-the-scales-how-smote-transforms-machine-learning-with-imbalanced-data-a6d3367254cd
https://machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/
https://machinelearningmastery.com/5-effective-ways-to-handle-imbalanced-data-in-machine-learning/
https://emilia-orellana44.medium.com/oversampling-3fd3fa662d32
https://machinelearningmastery.com/random-oversampling-and-undersampling-for-imbalanced-classification/
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html
https://www.geeksforgeeks.org/machine-learning/how-to-implement-stratified-sampling-with-scikit-learn/
https://www.geeksforgeeks.org/machine-learning/hyperparameter-tuning/?ref=hackernoon.com
https://machinelearningmastery.com/hyperparameter-optimization-with-random-search-and-grid-search/

# Tutorials

## Machine learning repo

https://github.com/mevol/DataScience_EBI

## Image augmentation Colab

https://colab.research.google.com/drive/18Ew7O8HnyhlUAkK9lNfwV9By3qTltlM6?usp=sharing

# Protein crystal classification

# The challenge

research papers

**IUCrJ**
ISSN: 2052-2525
BIOLOGY | MEDICINE

**Volume 13** | **Part 3** | May 2026 | Pages 235-248
https://doi.org/10.1107/S2052252526001399
OPEN ACCESS
Cited by 1

## AXIS: a Lab-in-the-Loop machine learning approach for automated detection of macromolecular crystals

**Aurelien Personnaz,[a] Sihyun Sung,[a] Raphael Bourgeas,[a] Sruthi Unni,[a] Florine Dupeux,[b] Bukunmi Adediran,[a] Rosicler Barbosa,[a] Anne-Sophie Humm,[a] Euan Colaco-Osorio[a] and José Antonio Márquez[a]***

[a]European Molecular Biology Laboratory, 71 Avenue des Martyrs, Grenoble 38000, France, and [b]Institut de Biologie Structurale, CNRS, 71 Avenue des Martyres, Grenoble 38000, France
*Correspondence e-mail: marquez@embl.fr

Edited by J. L. Smith, University of Michigan, USA (Received 7 November 2025; accepted 10 February 2026; online 19 March 2026)

Macromolecular crystallography provides mechanistic understanding of biological processes and can be applied in drug design. Nowadays, the use of robotic systems for crystal growth and diffraction analysis is widespread and high-throughput protein-to-structure pipelines for ligand and fragment screening are revolutionizing the field. However, the identification of crystals is still largely carried out through manual inspection, sometimes involving tens of thousands of images, which represents a bottleneck in an otherwise highly automated process. Here we describe AXIS, an AI-based Crystal Identification System combining the DINOv2 computer vision model, state-of-the-art transfer learning and MARCO, the largest crystallization dataset available to date, for automated crystal detection. AXIS can operate with both visible and UV light images and integrates a Lab-in-the-Loop approach combining ML and expert inputs for iterative learning and specialization. AXIS enables automated annotation of large crystallization image datasets with performance and accuracy comparable to that of human experts, and the Lab-in-the-Loop approach introduced here enables efficient adaptation to local conditions, facilitating widespread application, which has been a major limitation to date. AXIS can help to correct human errors in image annotation and removes critical bottlenecks, particularly in the context of extensive crystallization screens or high-throughput applications like fragment and ligand screening, unlocking the potential for higher levels of automation that are key in both fundamental and translational research.

**Keywords:** **structural biology**; **artificial intelligence**; **AI**; **machine learning**; **Lab-in-the-Loop**; **image processing**; **automation**; **high-throughput crystallography**.

**Similar articles** **Reuse permissions** **PowerPoint slides**

# Your target

| Model | Balanced accuracy (%) | Crystal recall (%) |
|---|---|---|
| AXIS-Vis-v1 | 93.1 | 90.0 |

| Model | Balanced accuracy (%) | Recall (%) |
|---|---|---|
| MARCO | 85.1 | 76.6 |
| CHiMP-v2 | 91.1 | 90.3 |
| AXIS-VMXi | 92.01 | 94.48 |

# Your data

training: https://zenodo.org/records/17279592 (5.6GB)

testing: https://zenodo.org/records/17279082 (2.4GB)

https://journals.iucr.org/m/issues/2026/03/00/jt5083/index.html

https://teachablemachine.withgoogle.com/